

SIMATS ENGINEERING**ASSESSMENT TOOL 1****Course Name: Cryptography and Network Security Course Code: CSA51**

Student Name	G. DAFNIE
Register No.	192521265
Course Outcome	CO1: Apply classical encryption techniques using symmetric and asymmetric ciphers to encrypt and decrypt data for ensuring data security (BL3, BL4)
Assessment Weightage	40%
Questions Attempted	Question 3 and Question 4

Annexure A – APPLICATION-BASED QUESTIONS

Code of Conduct: *I, G. Dafnie (Reg. No. 192521265), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

QUESTION 3

Scenario: Secure Messaging System. A company deploys a keyword-based encryption technique for internal communication. An employee sends the word “HELLO” using the keyword “KEY” to ensure confidentiality.

- Demonstrate how the encryption process transforms the message step-by-step.
- Explain how the repeating keyword influences each character of the ciphertext.

ANSWER

1. Aim

To encrypt the plaintext “HELLO” using the keyword “KEY” with a keyword-based polyalphabetic substitution technique (the Vigenère cipher), and to analyse how the repeating keyword influences every ciphertext character.

2. Concept Used

The Vigenère cipher is a classical symmetric polyalphabetic cipher. Instead of one fixed shift (as in the Caesar cipher), each plaintext letter is shifted by an amount decided by the corresponding letter of a repeating keyword. Letters are mapped to numbers A=0, B=1, ..., Z=25 and the operation is performed modulo 26.

$$\text{Encryption: } C_i = (P_i + K_i) \bmod 26$$

$$\text{Decryption: } P_i = (C_i - K_i + 26) \bmod 26$$

3. Step-by-Step Encryption Procedure

Step 1 – Letter-to-number mapping. Plaintext HELLO = 7, 4, 11, 11, 14. Keyword KEY = 10, 4, 24.

Step 2 – Repeat the keyword until it matches the length of the plaintext (5 letters): KEY is extended to K E Y K E.

Position	Plain letter (P)	Key letter (K)	P value	K value	(P + K) mod 26	Cipher letter (C)
1	H	K	7	10	17	R
2	E	E	4	4	8	I
3	L	Y	11	24	35 mod 26 = 9	J
4	L	K	11	10	21	V
5	O	E	14	4	18	S

Step 3 – Result. Plaintext: HELLO → Keyword stream: KEYKE → **Ciphertext: RIJVS**

4. Decryption (Verification)

The receiver, who knows the keyword KEY, reverses the process: $R(17)-K(10)=7=H$; $I(8)-E(4)=4=E$; $J(9)-Y(24)=-15 \rightarrow -15+26=11=L$; $V(21)-K(10)=11=L$; $S(18)-E(4)=14=O$. The original message HELLO is recovered, which confirms the encryption is correct.

5. Implementation (Python)

```
def vigenere(text, key, mode='enc'):
    out, key = [], key.upper()
    for i, ch in enumerate(text.upper()):
        p = ord(ch) - 65
        k = ord(key[i % len(key)]) - 65
        v = (p + k) % 26 if mode == 'enc' else (p - k) % 26
        out.append(chr(v + 65))
    return ''.join(out)

c = vigenere('HELLO', 'KEY', 'enc')
print('Ciphertext :', c)
print('Decrypted  :', vigenere(c, 'KEY', 'dec'))
```

Output: Ciphertext : RIJVS Decrypted : HELLO

6. Influence of the Repeating Keyword on Each Ciphertext Character

Plain letter	Key letter (shift)	Effect on ciphertext
H	K (shift 10)	H moves 10 places forward to R.
E	E (shift 4)	E moves 4 places forward to I.
L (1st)	Y (shift 24)	L moves 24 places forward, wraps around Z, and becomes J.
L (2nd)	K (shift 10)	The same letter L becomes V, because the key letter has changed.
O	E (shift 4)	O moves 4 places forward to S; the key restarts from its second cycle.

- **Different shift for every position:** the keyword supplies a shift pattern of 10, 4, 24, 10, 4, which repeats every 3 characters (the key length).
- **Repeated plaintext letters give different ciphertext letters:** the two L's in HELLO become J and V. This hides letter frequencies and defeats the simple frequency analysis that breaks a Caesar cipher.
- **Periodic repetition:** because the key is reused, the 1st and 4th characters share the shift K and the 2nd and 5th share the shift E. For long messages this periodicity can be exploited by the Kasiski examination and the Index of Coincidence, so a longer, non-repeating key gives better security.

7. Analysis and Conclusion

The keyword-based Vigenère technique gives HELLO $\rightarrow$ RIJVS. It is stronger than a single-shift Caesar cipher because each character uses a different substitution alphabet governed by the key, but a short repeating keyword such as KEY (length 3) remains vulnerable to cryptanalysis. The technique therefore demonstrates the principle of polyalphabetic substitution, while practical systems should use long, random keys (one-time pad) or modern ciphers such as AES.

QUESTION 4

Scenario: Suspicious Digital Asset Investigation. During a digital forensic audit, investigators suspect that a company logo file may contain hidden confidential information embedded within it. The file appears normal upon casual inspection.

- a. Analyze how investigators could detect the presence of hidden data within the image.
- b. Recommend two techniques that could strengthen the security of such hidden communications against detection or extraction.

ANSWER

1. Aim

To analyse how hidden data embedded in a company logo image can be detected, and to recommend techniques that make such hidden communication more resistant to detection and extraction.

2. Concept Used – Steganography

Steganography is the practice of concealing a secret message inside an ordinary carrier (image, audio, video or text) so that the very existence of the message is hidden. Unlike cryptography, which makes a message unreadable, steganography makes the message unnoticed. In images the most common method is Least Significant Bit (LSB) substitution, where the lowest bit of each pixel colour value (R, G, B) is replaced with a bit of the secret data. Changing the last bit alters a colour value by at most 1 out of 255, which is invisible to the human eye, so the logo looks normal on casual inspection. Data may also be appended after the file's end marker or placed in metadata fields.

3. Part (a) – Detecting Hidden Data in the Image

Investigators follow a systematic forensic workflow, moving from file-level checks to pixel-level statistical analysis:

Step	Detection method	What the investigator does / looks for	Tools
1	Preserve evidence and hash	Create a forensic copy and compute SHA-256 hashes. Compare with the original logo in the brand repository; a hash mismatch indicates modification.	sha256sum, FTK Imager
2	File-format and size analysis	Verify that the real file type matches the extension (magic bytes). Check whether the file size is unusually large for its resolution and compression.	file, hexdump, xxd
3	Metadata and string inspection	Examine EXIF/comment fields and extract printable strings for readable text, keys or passwords.	exiftool, strings

Step	Detection method	What the investigator does / looks for	Tools
4	Trailing/embedded data check	Look for data after the end-of-file marker (JPEG FFD9, PNG IEND) or embedded archives and signatures.	binwalk, hex editor (HxD)
5	Visual and bit-plane analysis	View the LSB planes of each colour channel. Natural images show noise-like patterns; embedded data produces structured or unnaturally random regions.	StegSolve, zsteg
6	Statistical steganalysis	Apply the Chi-square attack, RS analysis and sample-pair analysis. LSB embedding equalises the frequency of pixel-value pairs, which these tests detect.	StegExpose, StegDetect, Python (NumPy)
7	Histogram comparison	Compare histograms of the suspect and original image; embedding creates the “pairs of values” flattening effect.	Python (OpenCV, Matplotlib)
8	Extraction attempt	If indicators are found, try known extraction tools and password dictionaries to recover the payload.	steghide, OpenStego, stegcracker

Illustration of the LSB principle: a pixel value 10110100 (180) carrying secret bit 1 becomes 10110101 (181). The visual difference is negligible, but across thousands of pixels the statistical distribution of the least significant bits changes, which is what steganalysis tools detect.

Sample detection script (LSB bit-plane check):

```
from PIL import Image
import numpy as np

img = np.array(Image.open('logo.png').convert('RGB'))
lsb = img & 1 # extract least significant bits
ratio = lsb.mean() # natural images: not exactly 0.5 in all areas
print('LSB ones ratio:', round(float(ratio), 4))

# Save the LSB plane for visual inspection
Image.fromarray((lsb[:, :, 0] * 255).astype('uint8')).save('lsb_red.png')

# Compare with the original logo
orig = np.array(Image.open('original_logo.png').convert('RGB'))
diff = (img != orig).sum()
print('Different pixel values:', diff)
```

Interpretation of results: an LSB ratio very close to 0.5 across the whole image, a noise-like LSB plane, differing pixels from the original logo, a mismatching hash, or text recovered by

strings/binwalk provide strong evidence of hidden content. A clean image shows structured LSB planes that follow the logo's shapes and no trailing data.

4. Part (b) – Two Techniques to Strengthen Hidden Communication

Technique 1: Encrypt before embedding (encrypt-then-hide). The secret message is first encrypted with a strong symmetric cipher such as AES-256 (with a key derived from a strong passphrase) and only then embedded in the image. Even if investigators detect and extract the hidden bits, they obtain only ciphertext that is unreadable without the key. Encrypted data also looks statistically random, which makes the embedded payload more uniform and harder to distinguish from natural noise. This provides two layers of protection: secrecy of the content (cryptography) and secrecy of the existence (steganography).

Technique 2: Key-based random and adaptive embedding in a transform domain. Rather than writing bits sequentially into the first pixels, the embedding positions are chosen by a pseudo-random number generator seeded with a secret stego-key, so only someone with the key knows where the data lies. The data is placed in complex, high-texture or edge regions of the image where changes are least noticeable, using a low embedding rate and, where possible, transform-domain methods such as DCT/DWT coefficients or LSB matching (± 1), instead of plain LSB replacement. This avoids the pairs-of-values pattern, reduces the effect of Chi-square and RS analysis, and makes extraction without the key practically impossible.

Aspect	Technique 1: Encrypt-then-embed	Technique 2: Random adaptive embedding
Main protection	Content confidentiality	Resistance to detection and location
Defeats	Reading of extracted data	Statistical steganalysis, sequential extraction
Secret needed	Encryption key (e.g., AES)	Stego-key (PRNG seed)
Example	AES-256 then LSB embed	Key-seeded pixel selection, DCT/DWT, LSB matching

5. Analysis and Conclusion

A logo file that looks normal can still conceal confidential data through LSB steganography, appended bytes or metadata. Investigators can detect it through hash comparison, file-structure and metadata inspection, bit-plane visualisation and statistical steganalysis, supported by tools such as exiftool, binwalk, StegSolve and StegExpose. For senders, the most effective strengthening measures are combining encryption with steganography (so extracted data is useless) and using key-based, adaptive, low-rate embedding (so the data is harder to find). Combining both techniques gives layered security: attackers must first detect the hidden payload, then locate it, and finally break the encryption.

6. References

- [1] Alabady, et al. (2025). Enhanced Vigenere cipher algorithm for improved cryptographic security. *Quantum Journal of Engineering, Science and Technology*, 6(1), 1–12.
<https://doi.org/10.55197/qjoest.v6i1.194>
- [2] Enhancing the Vigenère cipher for Unicode compatibility: A modern approach to polyalphabetic encryption. (2025). *Journal of Institutional Research, Big Data Analytics and Innovation*, 1(4), 39–54.
<https://doi.org/10.5281/zenodo.17666872>
- [3] De La Croix, N. J., Ahmad, T., & Han, F. (2024). Comprehensive survey on image steganalysis using deep learning. *Array*, 22, 100353. <https://doi.org/10.1016/j.array.2024.100353>
- [4] A comprehensive survey of digital image steganography and steganalysis. (2024). *APSIPA Transactions on Signal and Information Processing*, 13(1), 1–67.
- [5] Kuznetsov, A., Luhanko, N., Frontoni, E., Romeo, L., & Rosati, R. (2024). Image steganalysis using deep learning models. *Multimedia Tools and Applications*, 83, 48607–48630.
<https://doi.org/10.1007/s11042-023-17591-0>
- [6] Kheddar, H., Hemis, M., Himeur, Y., Megías, D., & Amira, A. (2024). Deep learning for steganalysis of diverse data types: A review of methods, taxonomy, challenges and future directions. *Neurocomputing*, 127528. <https://doi.org/10.1016/j.neucom.2024.127528>

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation